Desarrollo del *inflight software* para la OBC secundaria del Satélite Quetzal-2

Flavio André Galán Donis



UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



Desarrollo del *inflight software* para la OBC secundaria del Satélite Quetzal-2

Trabajo de graduación presentado por Flavio André Galán Donis para optar al grado académico de Licenciado en Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Guatemala, octubre

2026

Vo.Bo.:

(f) _____

Ing. Kuk Ho Chung

Tribunal Examinador:

(f) _____

Ing. Kuk Ho Chung

(f) _____

MSc. Carlos Esquit

(f) _____

Ing. Luis Pedro Montenegro

Fecha de aprobación: Guatemala, _____ de _____ de 2026.

Índice

Resumen 6
Abstract 7
1 Introducción 1
2 Justificación 3
3 Objetivos 4
3.1 Objetivo general 4
3.2 Objetivos específicos 4
4 Marco Teórico 5
4.1 Quetzal 1 y 2 5
4.2 Computadora a Bordo (OBC) 6
4.3 Comunicación Serial: I2C y UART 7
4.3.1 UART 8
4.3.2 RS485 8
4.3.3 I2C 9
4.4 Protocolo de *Handover* 10
4.5 Sistemas Operativos 12
4.6 Arquitectura de Software 12
4.6.1 Fiabilidad en *Software* 14
4.6.1.1 Fuzzing 14
4.6.1.2 LLMs 15
5 Metodología 17
5.1 Ambiente de Desarrollo 17
5.2 Configuración del Hardware 17
5.2.1 I2C 17
5.2.2 UART 18
5.3 OBC Secundaria Quetzal-2 20
6 Resultados 22
7 Discusión de Resultados 25
8 Conclusiones 27
Bibliografía 28
9 Anexos 31
9.1 Repositorio de Código 31
Glosario 32

Lista de figuras

Figura 1 Prototipo conceptual de Quetzal-2. Fuente: Universidad del Valle de Guatemala. 5
Figura 2 *CubeSat* Quetzal 1. Fuente: Universidad del Valle de Guatemala. 6
Figura 3 Diagrama de componentes de la OBC y sus protocolos de comunicación. 7
Figura 4 Diagrama de conexión de pines entre dos dispositivos que soportan el protocolo de comunicación UART. 8
Figura 5 Diagrama de conexión de pines entre dos dispositivos que soportan el protocolo de comunicación UART. 9
Figura 6 Diagrama de conexión de pines entre dos dispositivos que soportan el protocolo de comunicación I2C. 10
Figura 7 Diagrama de secuencia del protocolo de *handover* para un caso general. 11
Figura 8 Árbol de ejecución de un programa simple. Nótese como cada función genera su propio bloque y algunas ramas tienen condiciones que deben cumplirse para ser ejecutadas. 15
Figura 9 Comunicación I2C entre computadora primaria y secundaria. 18
Figura 10 Comunicación UART entre OBC y módulos del satélite. 19
Figura 11 Máquina de Estados Finitos (incompleta) de la OBC secundaria del Quetzal-2. Elaboración propia. 21
Figura 12 Arquitectura de la OBC secundaria. Elaboración propia. 21
Figura 13 Prueba del protocolo de handover corriendo localmente dentro de una misma computadora. 22
Figura 14 Prueba del protocolo de handover corriendo con hardware real entre la computadora primaria y secundaria. *Logs* desde la perspectiva de la OBC secundaria. 22
Figura 15 Prueba del protocolo de handover corriendo con hardware real entre la computadora primaria y secundaria. Realizado durante la visita de David Cuartielles, cofundador de Arduino y Andrés Sabas, fundador de Electronic Cats. 23
Figura 16 Fotografía tomada con la *Payload MILO* durante la prueba de handover. . . 24
Figura 17 Estados de la OBC secundaria de estados finitos. 25

Lista de cuadros

Cuadro 1 Tabla de verdad del protocolo RS485. 9

Resumen

El Quetzal-2 es un nanosatélite CubeSat 2U desarrollado como proyecto académico por estudiantes y *staff* de la Universidad del Valle de Guatemala el cual pone a prueba una *On-Board Computer* (OBC) diseñada localmente que ejecuta una inteligencia artificial capaz de identificar nubes. Este nanosatélite cuenta con una computadora a bordo compuesta de por dos unidades de procesamiento, una OBC principal basada en hardware comercial de GomSpace y otra OBC secundaria desarrollada localmente utilizando Portentas H7 de Arduino.

Las OBC son sistemas vitales dentro de una misión espacial puesto que gestionan todos los subsistemas del satélite así como la comunicación entre estos. Este proyecto se centra en las bases de esta OBC local a nivel de *software*, sentando las bases para los distintos modos de operación que necesita la misión, la toma de fotografías y la interoperabilidad entre las dos unidades de procesamiento que componen la OBC completa.

El impacto de esta OBC desarrollada localmente representa un gran punto de inicio para próximas misiones espaciales, bajando aún más la barrera económica de entrada para desarrollar misiones espaciales con respecto a soluciones *of-the-shelf*, puesto que demuestra que no es necesario el uso de *hardware* altamente costoso con muchas más protecciones para llevar a cabo una misión exitosa.

Abstract

This is an abstract of the study developed under the

Quetzal-2 is a CubeSat 2U nanosatellite developed as an academic proyect by students and staff from the *Universidad del Valle de Guatemala*, this missions tests an On Board Computer (OBC) designed locally that executes an artificial inteligence capable of identifying clouds in pictures. This nanosatellite has an OBC made up of two processing units, the primary processing unit is based on commercial technology from GomSpace, the secondary OBC is developed locally and is powered by Portentas H7 from Arduino.

The OBCs are vital systems in a space mission because they manage every other subsystem from the satellite and the communication between them. This project focuses in the bases of the secondary OBC developed locally at a software level, building the bases for the different operation modes the mission requires, the capture of photography and the interoperability of the two processing units of the OBC.

The impact of this OBC developed locally represents a huge advancement on the starting point for next space missions, lowering the entry cost for developing space missions with respect to of-the-shelf solutions, since it proves that the use of space specialized hardware, which is relatively highly expensive is not a requirement for a successful mission.

I

Introducción

El Quetzal-2 es un proyecto académico desarrollado por estudiantes y personal de la Universidad del Valle de Guatemala la cual pone a prueba una computadora a bordo (OBC por sus siglas en inglés) diseñada en UVG capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales [1]. Este proyecto es importante no solo porque representa un avance tecnológico en Guatemala, sino también por su impacto dentro de la juventud del país, inspirando a futuros científicos, ingenieros e innovadores [2].

La misión espacial es un CubeSat 2U, 10x20x10cm y con un peso máximo de 4kg [3] y cuenta con 4 objetivos específicos que busca completar, llamadas cargas útiles (*Payloads* en inglés). Para este proyecto nos interesa específicamente el *inflight software* de la OBC secundaria de Quetzal-2 y su interacción con *Payload MILO*, el subsistema encargado del reconocimiento de nubosidad y la toma de fotografías en el espacio [1]. Es importante notar que estos 4 objetivos de misión son independientes de los 3 objetivos específicos de este trabajo profesional detallados en la sección de objetivos.

Las OBCs cumplen un papel clave dentro de los subsistemas del satélite puesto que son las encargadas de manejar todas las tareas, intercambio de información entre módulos y la colecta de información sobre los demás subsistemas (*housekeeping*) antes de la conexión con la estación en tierra [4].

Algunos de los subsistemas que debe manejar una OBC son:
- Un sistema de poder (EPS por sus siglas en inglés).
- Un sistema que determina la altidud (ADCS por sus siglas en inglés).
- Un sistema de comunicación utilizando radiofrecuencias.
- Una o más cargas útiles, por ejemplo una cámara o transmisor de señales junto con su controlador.

La inicialización, intercomunicación y gestión de todos estos subsistemas recae sobre la OBC [5]. Debido a las altas limitaciones energéticas en satélites de esta escala se debe tener especial cuidado con el presupuesto de energía que se le puede brindar a cada uno de los subsistemas. De esta necesidad nacen los modos de operación, los cuáles restringen las funcionalidades del satélite según la tarea para la que se quiere que se especialice en ese momento.

Este trabajo profesional busca desarrollar un sistema de *software* para la OBC diseñada localmente (*in-house*) capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de *handover* con la computadora principal, validado en un entorno controlado terrestre. Para el desarrollo de este sistema se utiliza una metodología propia

del laboratorio espacial UVG, basada en la metodología de ingeniería de sistemas de la NASA [6].

II

Justificación

La misión espacial Quetzal-2 al ser una misión académica y no un proyecto de gobierno o una multinacional como Tesla, cuenta con recursos económicos relativamente limitados. Esto genera una necesidad real de reducir costos en donde sea posible, por lo tanto, un *inflight software* diseñado localmente, adaptado a las necesidades específicas de la misión pero reutilizable en misiones futuras y además es capaz de ejecutarse dentro de componentes más baratos resulta atractivo, dando origen a este proyecto.

En cuanto a las tecnologías a utilizar, las OBCs dependen en gran medida del microcontrolador seleccionado para su misión [7]. En el Quetzal-2 se utiliza una pareja de PortentasH7 Lite para la OBC secundaria, éstas cuentan con dos procesadores ARM diseñados por el proveedor STM32 [8]. Por lo tanto, nuestras opciones se encuentran limitadas a lo que este proveedor pueda ofrecer.

Según la literatura actual, el uso de *Real Time Operating Systems* (RTOS por sus siglas en inglés) representa una práctica común para escribir sistemas complejos en el mundo del *embedded software*, incluyendo en misiones espaciales como el STUDSAT-2 [7]. Para este proyecto se utiliza FreeRTOS debido a su soporte para PortentaH7 y su comunidad de desarrolladores, la cual es grande y activa [9]. Debido a que FreeRTOS expone un API oficial en C para su desarrollo [9] y a los bajos márgenes de presupuesto de energía con los que se cuenta en misiones de escalas similares a Quetzal-2 [4], el lenguaje que se utiliza para el desarrollo del *inflight software* es C.

III

Objetivos

3.1 Objetivo general

Desarrollar localmente las bases de un *inflight software* para la *On Board Computer* (OBC) secundaria diseñada localmente para el Quetzal-2, capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de *handover* con la OBC principal, validado en un entorno controlado terrestre.

3.2 Objetivos específicos

1. Integrar el módulo de cámara al sistema de *software* de la OBC secundaria, logrando la captura de la imagen y transmisión de informaciones de la imagen desde la carga útil MILO hacia la OBC secundaria desarrollada localmente.
2. Desarrollar el sistema base de gestión de modos de operación del *software*, implementando tres modos: Arranque, Toma de fotografía y Nominal preliminar.
3. Desarrollar un Producto Mínimo Viable (MVP) del sistema de *handover* entre la OBC principal y la OBC secundaria diseñada localmente.

IV

Marco Teórico

4.1 Quetzal 1 y 2

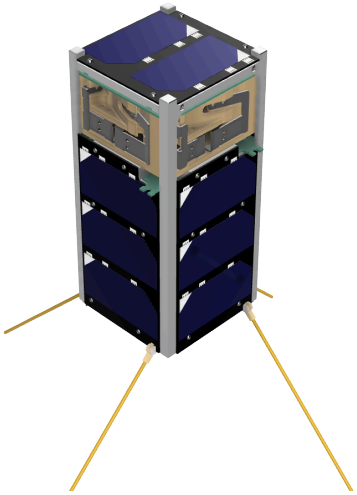


Figura 1: Prototipo conceptual de Quetzal-2. Fuente: Universidad del Valle de Guatemala.

El Quetzal-2 (Figura 1) es el segundo proyecto aeroespacial de la Universidad del Valle de Guatemala, el cual es desarrollado por estudiantes y personal académico de la institución [1].

Su predecesor, el Quetzal-1 (Figura 2), fue el primer satélite guatemalteco, cuya misión buscaba probar un sensor multiespectral para adquirir información remota para conservar recursos naturales de forma independiente [2]. El trabajo realizado para Quetzal-1 representa las bases para el nuevo y mejorado Quetzal-2, el cual busca poner a prueba una computadora a bordo (OBC por sus siglas en inglés) diseñada localmente, capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales. Así como validará un subsistema para desorbitación responsable y permitirá transmitir datos satelitales en tiempo real a centros educativos del país [1].



Figura 2: *CubeSat* Quetzal 1. Fuente: Universidad del Valle de Guatemala.

Debido a la complejidad de la misión, Quetzal-2 es un CubeSat 2U, el doble de tamaño que su predecesor [1]. CubeSat es un estándar que inició en 1999, desarrollado por el profesor Jordi Puig-Sauri y Bob Twiggs. La intención de este estándar es reducir costos y tiempos de desarrollo, al mismo tiempo que incrementa la accesibilidad al espacio. Todos los satélites CubeSat adoptan un tamaño y peso medido en unidades ("U"), la medida que define el estándar. Un CubeSat de 1U (como el Quetzal-1 [2]) es un centímetro de 10cm de lado con una masa de hasta 2kg [3].

4.2 Computadora a Bordo (OBC)

Esta clase de misiones espaciales se componen de varios subsistemas comunicándose entre sí para llevar a cabo el objetivo de la misión, de esta realidad surge la necesidad de una OBC, la cual se encarga de manejar y dirigir de forma autónoma estas comunicaciones, así como la interacción con la estación de control terrena (GCS por sus siglas en inglés) [4].

Según Gildeh [10], un CubSat típico se compone de varios subsistemas, como mínimo:

- Sistema de poder (EPS por sus siglas en inglés), cargado por paneles solares.
- Una carga útil (comúnmente llamada *payload*), el/los objetivo(s) de la misión, generalmente una cámara o similar.
- Sistema de control y determinación de altura (ADCS por sus siglas en inglés).
- Sistema de comunicación por radiofrecuencia.
- Una OBC que permite la comunicación entre todos estos subsistemas.

En el caso de Quetzal-2, la OBC realmente se divide en 2, una principal y una secundaria. La OBC principal, es la misma que se utilizó en Quetzal-1, la Gomspace Nanomind A3200 [11], esta computadora fue especialmente diseñada para misiones espaciales, como sensores integrados para control de altitud y protecciones especiales contra radiación [12].

La OBC secundaria, la cual es diseñada localmente, está compuesta por una pareja de Portentas H7 Lite, ambas cuentan con dos cores de procesamiento y 8MB de SDRAM [8]. La OBC secundaria, al igual que la primaria, necesita gestionar la comunicación entre los distintos subsistemas, por lo que se le considera el cerebro del satélite [4] y según [13] necesita cumplir con las siguientes tareas:

- Grabar y guardar la telemetría del satélite, incluyendo la data de las *payloads* y su transmisión hacia la GCS.
- Cifrado y descifrado de los paquetes de datos enviados y recibidos de la GCS.
- Monitoreo y gestión de subsistemas, incluso reiniciando los que sean necesarios.

Generalmente, la OBC no se encarga de el manejo de energía, en su lugar un subsistema por aparte (el EPS) se encarga de apagar y reinciar sistemas dentro del satélite. Aunque sí hay casos en donde tal tarea se le atribuye a la OBC [14]. La Figura 3 muestra cómo se organizan los componentes de la OBC de Quetzal-2 así como la forma de comunicación con otros submódulos:

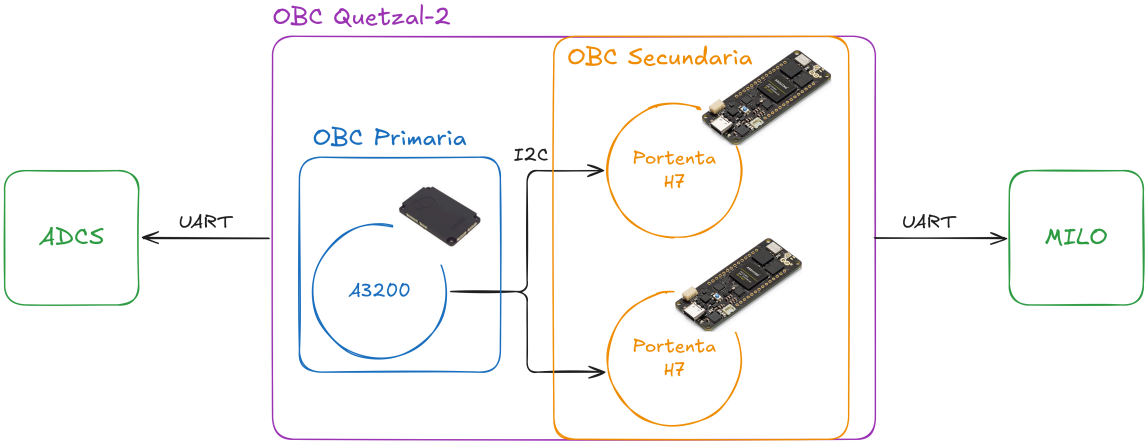


Figura 3: Diagrama de componentes de la OBC y sus protocolos de comunicación.

4.3 Comunicación Serial: I2C y UART

Existen varias interfaces seriales de comunicación que se pueden utilizar para la intercomunicación de microcontroladores, muchas aunque desarrolladas para una aplicación en específico se han vuelto universales, RS485 e I2C caen en esta categoría [15]. Dentro del Quetzal-2 se utilizan estos dos protocolos para la comunicación interna entre sus subsistemas, RS485 es especialmente popular debido a que permite conectar múltiples puntos de control utilizando un bus serial, además de ser un protocolo de comunicación probado en producción por años lo que lo hace una opción segura cuanto menos [16]. Mientras que I2C facilita la comunicación entre microcontroladores utilizando un modelo de maestro esclavo, en donde solo el maestro puede iniciar la comunicación [17].

Dentro de Quetzal-2, los protocolos se utilizan para los siguientes propósitos:

- RS485: Comunica la OBC primaria y secundaria con el resto de subsistemas del satélite, es decir: *Payload* MILO, *ADCS*, *ADM*, etc. Se utiliza el protocolo de *hardware* UART para su implementación.
- I2C: Se reserva únicamente para la comunicación entre la OBC primaria y la OBC secundaria, siendo la OBC primaria la maestra y la OBC secundaria la esclava. Esta decisión es clave para el funcionamiento de la arquitectura de *handover*.

4.3.1 UART

El protocolo *Universal Asynchronous Receiver-Transmitter* (UART) es uno de los protocolos de hardware más usados para comunicación entre dispositivos. El protocolo necesita de dos pines: uno para recibir información, llamado comúnmente `RX` y otro para enviar información, llamado `TX`. Ambos se conectan de forma cruzada entre los dispositivos para lograr así la recepción y comunicación de datos en ambos dispositivos [18], como se ve en la Figura 4.

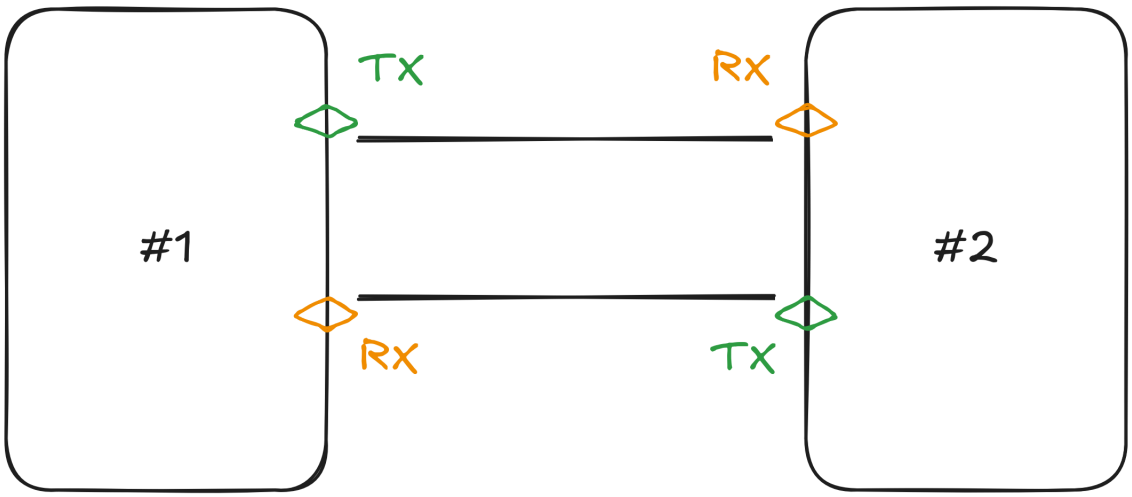


Figura 4: Diagrama de conexión de pines entre dos dispositivos que soportan el protocolo de comunicación UART.

4.3.2 RS485

El protocolo RS485 se basa en el protocolo RS422, su principal ventaja con respecto a RS422 es que permite la conexión de hasta 32 dispositivos (tanto transmisores como receptores) mientras que RS422 solamente admite 10 receptores y un transmisor. RS485 hereda de RS422 su alta resistencia al ruido, ya que utiliza una corriente diferencial entre dos cables para transmitir la señal, cuya conexión se ve en la Figura 5. Lo que significa que un byte que se transmite no es un pulso de corriente dentro de un solo cable, sino una diferencia de corriente entre ambas líneas de conexión del protocolo, el Cuadro 1 muestra una tabla de verdad con la entrada y cómo se transmite por los cables. Al protocolo solamente le

interesa la diferencia entre ambas líneas, por lo que mientras pueda interpretar a una línea como 0 y a otra como 1 seguirá funcionando, lo que lo hace altamente resiliente a ruido [19].

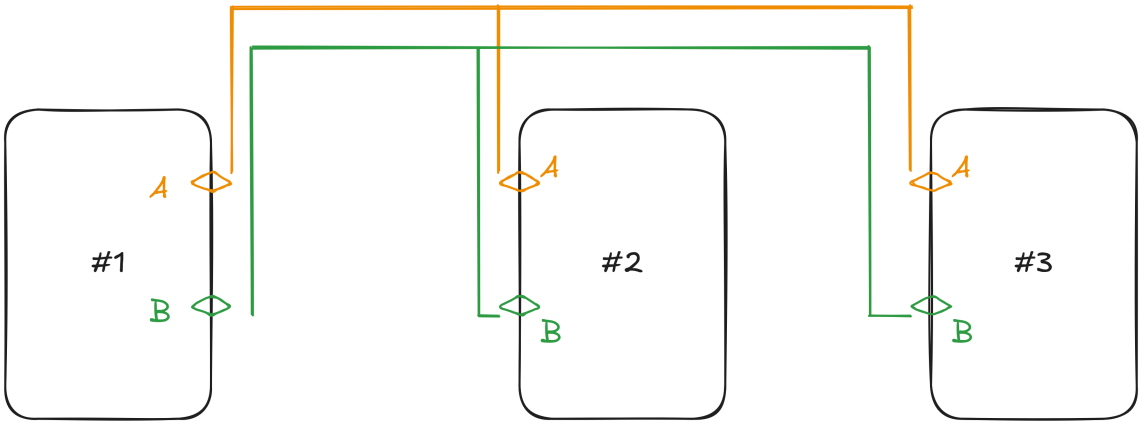


Figura 5: Diagrama de conexión de pines entre dos dispositivos que soportan el protocolo de comunicación UART.

Entrada	**A**	**B**	**Salida**
0	1	0	0
1	0	1	1

Cuadro 1: Tabla de verdad del protocolo RS485.

4.3.3 I2C

Al igual que RS485, I2C es un protocolo de comunicación serial entre dispositivos, la transmisión de datos es serial y sincrónica, lo que implica el paso de un pulso de reloj para sincronizar la salida y entrada de los datos entre los dispositivos. Requiere de dos líneas de conexión [17]:

1. SDA (*System Data*): Se utiliza para transmitir los datos entre los dispositivos.
2. SCL (*System Clock*): Se utiliza para transmitir los pulsos del reloj. Sincronizando el sistema.

Un ejemplo de conexión se puede apreciar en la Figura 6:

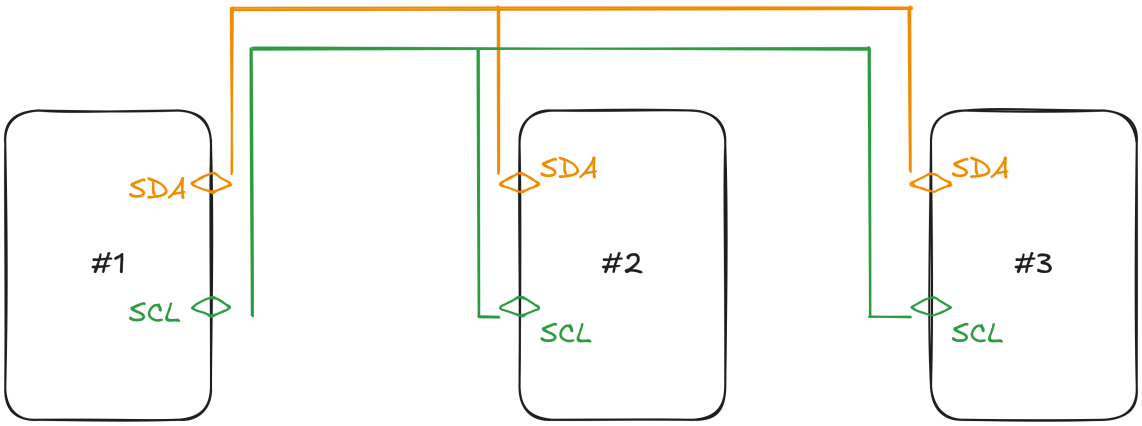


Figura 6: Diagrama de conexión de pines entre dos dispositivos que soportan el protocolo de comunicación I2C.

Un detalle importante dentro de este protocolo es el rol que cada dispositivo tiene dentro de la comunicación. Siempre se tiene un dispositivo maestro, encargado de dirigir la comunicación y otros dispositivos esclavos. Estos roles pueden alternar libremente durante *conversaciones* de dispositivos, es posible que para una rutina un microcontrolador sea maestro y para otra sea el esclavo [17].

4.4 Protocolo de *Handover*

El protocolo de *handover* es un sistema interno de la OBC primaria, diseñado localmente, cuyo objetivo principal es trasladar el control del satélite de sí misma a la OBC secundaria de una forma segura y redundante a fallos. Quetzal-2 no es la única misión que ha integrado más de una OBC en su sistema de mando, SHEFEX III, una misión alemana del 2017 también usó dos OBCs para redundancia, sin embargo su estrategia nunca fue ceder control de una OBC a la otra sino tener un backup en caso alguna de la dos fallara [20]. Otro ejemplo es la arquitectura DHS propuesta en 2023 para misiones académicas como profesionales [21]. Esta arquitectura en específico también se usó en el mismo año para otra misión espacial, la misión HERA, de la misma forma su principal objetivo era redundancia total, pero además buscaba autonomía operacional y alta capacidad de almacenamiento a bordo [22].

Una descripción general del protocolo se puede ver en la Figura 7. En donde se tiene a los 3 actores principales: A3200 representando la computadora principal, Portenta H7, representando la portenta que está activa en ese momento y SAMD, representando a el módulo que se necesita para una tarea, por ejemplo MILO.

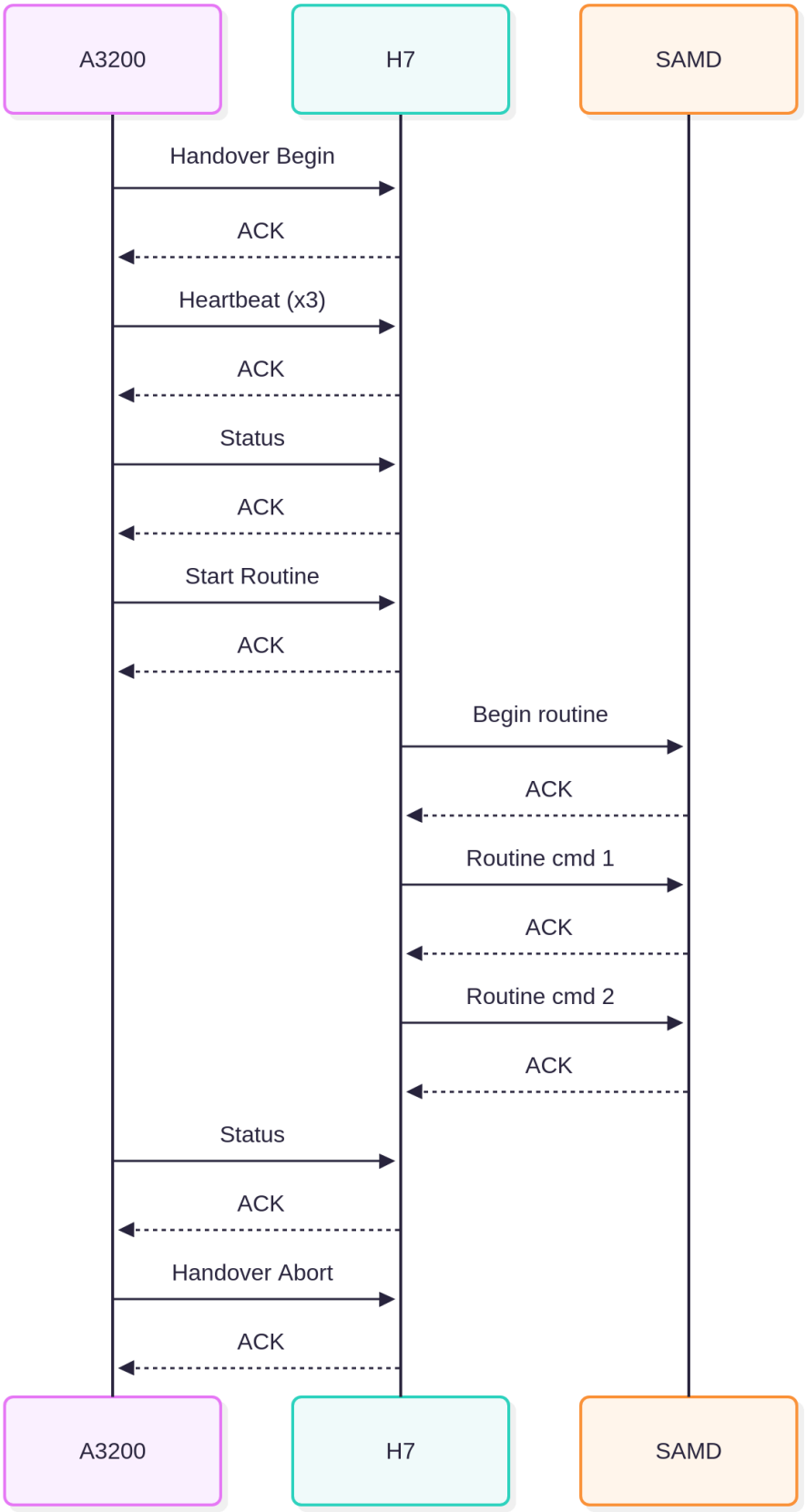


Figura 7: Diagrama de secuencia del protocolo de *handover* para un caso general.

En caso de fallos, el comando de `Handover abort` se enviará de forma automática por la OBC principal, además de que se tendrá un timer que permitirá la toma de control de parte de la OBC primaria en caso la secundaria deje de responder.

La OBC diseñada localmente es desarrollada en un lenguaje a bajo nivel, particularmente C, debido a que es el lenguaje principal en el que el *driver* a bajo nivel de STM32 es proveído, ofreciendo un mayor control sobre el uso de los recursos, casi tan granular como querramos [23]. Debido a la complejidad del sistema interno que debe manejar la OBC para gestionar la comunicación entre todos los subsistemas del satélite, se necesita un sistema operativo [4].

4.5 Sistemas Operativos

Existen muchos tipos de sistemas operativos (OS por sus siglas en inglés), según su infraestructura interna podemos tener sistemas operativos monolíticos, por capas, micro-kernels, módulos o híbridos siendo alguna combinación entre ellos [24]. Para propósitos del Quetzal-2, resulta de mayor importancia cómo el OS calendariza sus tareas que el cómo se compone internamente el sistema operativo como tal, ya que las misiones espaciales tienden a tener presupuestos muy ajustados tanto de memoria como procesamiento [4].

Los sistemas operativos en tiempo real (RTOS por sus siglas en inglés) fueron creados justamente para los casos en donde se tienen requerimientos de tiempo rígidos [24] y por esta razón se evaluaron distintas alternativas de implementación de un RTOS. RODOS, un sistema operativo de código abierto desarrollado por Sergio Montenegro, con un énfasis en facilidad de uso, rendimiento y probado en aplicaciones espaciales reales [25]; FreeRTOS, otro sistema operativo de código abierto, con una gran comunidad, excelente rendimiento y años de uso en producción [9]. Luego de unas pruebas de *porting* de RODOS a STM32H7, la arquitectura de la Portenta H7 Lite, se decidió a utilizar FreeRTOS por su alta compatibilidad con este *hardware*.

4.6 Arquitectura de Software

La arquitectura de software busca gestionar la comunicaciónn entre los componentes de software que conforman un sistema [26]. En este caso, sistema se referiría a la OBC diseñada localmente. Al ser una misión espacial, el acceso que tendremos al satélite es muy limitado, no podremos realizar parches de actualización de código por ejemplo. Por lo que tener la mayor garantía que podamos de que el sistema funciona y es resiliente a fallos es la prioridad más alta.

La *National Aeronautics and Space Administration* (NASA) es famosa por sus 10 reglas para desarrollar código crítico seguro:

1. **No usar recursión, saltos ni *goto statements***. La justificación de la NASA es debido a que simplificar el flujo de ejecución de nuestro programa generalmente se traduce a mejor claridad del código, así como en una mejora considerable de análisis estático.
2. **Todos los ciclos deben tener un límite superior fijo**. Sin recursión, y con ciclos que siguen un límite fijo de iteraciones se previene en su totalidad rutinas que se ejecutan de forma indefinida. Esta regla no aplica para rutinas que deben ser forzosamente de terminación indeterminada, como la espera de nuevos comandos de la estación terrena (GCS por sus siglas en inglés).
3. **No alojar memoria dinámica luego de la inicialización**. La alojación de memoria dinámica impacta negativamente en el rendimiento del sistema, además de que tiene un comportamiento notablemente difícil de predecir con análisis estático. Esta regla fuerza al programador a usar memoria estática alojada al declarar variables en el stack, si se combina con las reglas anteriores el código termina siendo legible y sencillo, lo que es excelente para analizadores estáticos que buscan probar que un programa se comportará como esperamos.
4. **No más de 60 líneas de código por función**. Funciones extremadamente largas son una señal de código pobremente arquitecturizado.
5. **Cada función debe tener un promedio de dos *assertions***. La NASA recomienda utilizar aserciones en el código para probar condiciones de ejecución tanto antes como después de correr una función. Una aserción siempre revisa una condición, y detiene el programa en caso de que la condición no se cumpla, son especialmente útiles si se combinan con pruebas unitarias.
6. **Declara todos los objetos de tipo data en el tamaño mínimo posible de *scope***. Esta regla simplifica la depuración del código cuando ocurre una falla, puesto que es obvio que una variable que no se encuentra en el scope del código que falla no tiene ninguna relación con el error en sí. Por lo tanto queremos minimizar la cantidad de variables dentro del scope de cada rutina, limitando así, los lugares en donde se puede corromper estado y afectar en la ejecución de nuestro programa.
7. **Todos los parámetros de retorno deben ser revisados por la función que los llamó**. **Todas las funciones deben revisar la validez de los parámetros proveídos**. Las interfaces con las que interactuamos los programadores están plagadas de funciones que puede que funcionen como esperamos o no, generalmente tienen una forma de reportar un error. Si una función debe ejecutarse solamente cuando el valor de retorno de otra función es exitoso, entonces es indispensable que este valor de retorno se revise, ese es el corazón de esta regla.
8. **El uso del preprocesador de C debe estar limitado a inclusión de *header files* y definiciones sencillas**. El preprocesador de C puede fácilmente ofuscar el código y confundir a analizadores estáticos, por lo que la NASA no recomienda su uso más allá de inclusiones sencillas.
9. **Solamente es permitido un nivel de referencia con punteros**. Según la NASA, los punteros son fácilmente usados de forma incorrecta, incluso por programadores experimentados, pueden hacer que seguir el flujo de control de un programa sea complicado y reestringen las capacidades que tienen los analizadores de código estático.

10. **Todo el código debe ser compilado desde el primer día de desarrollo con todas las alertas del compilador en la configuración más pedante. El código debe compilar sin advertencias**. Según la NASA, no hay ninguna excusa por la que una *warning* debería ser ignorada, incluyendo los falsos positivos. El código debe ser reescrito para facilitar su análisis estático y no confundir a la herramienta [27].

4.6.1 Fiabilidad en Software

El espacio no es el único sector dentro del mundo del *software* que necesita una resiliencia extraordinaria. Las bases de datos, son piezas de *software* que deben funcionar incluso si el disco se corrompe, como es el caso de TigerBeetle [28]. U otra base de datos venerada por su resiliencia a lo largo de los años, SQLite, con más de 1 billón de instancias en uso actualmente [29].

Aunque hechas para casos de uso muy distintos, ambas llegaron a la misma conclusión, la forma de garantizar resiliencia a fallos, incluso en ambientes hostiles, es utilizar el poder de la misma computadora para revisar tu código. No hablan de IA, sino de *fuzz testing*, *unit testing* y otra gran variedad de *xxx testing*. No basta solo el análisis estático, hay que poder garantizar de forma automatizada que la solución funciona y es resiliente a fallos, no porque el *linter* no encuentre errores, sino porque años de simulación que ocurren en horas o días en tiempo real lo respaldan [28], [29].

4.6.1.1 Fuzzing

Richard Hipp, uno de los creadores de SQLite, cuenta en su experiencia como desarrollador de este software que al alcanzar *100% branch coverage*, dejó de recibir reportes de bugs de sus usuarios por varios años [29]. Para entender esta métrica es importante entender primero qué es una «rama». En programación se asume que todas las instrucciones ejecutan una luego de la otra, sin embargo esto no siempre es cierto, es común en aplicaciones que ciertas instrucciones sean ejecutadas solamente cuando ciertas condiciones se cumplen. Esto genera caminos de instrucciones por los que el procesador puede o no pasar, todo el conjunto de estos caminos generan un árbol de posibles ejecuciones del programa en donde, según qué condiciones se cumplan, el procesador ejecutará una rama u otra [30]. La Figura 8 demuestra cómo se vería este árbol de ejecución para un programa sencillo.

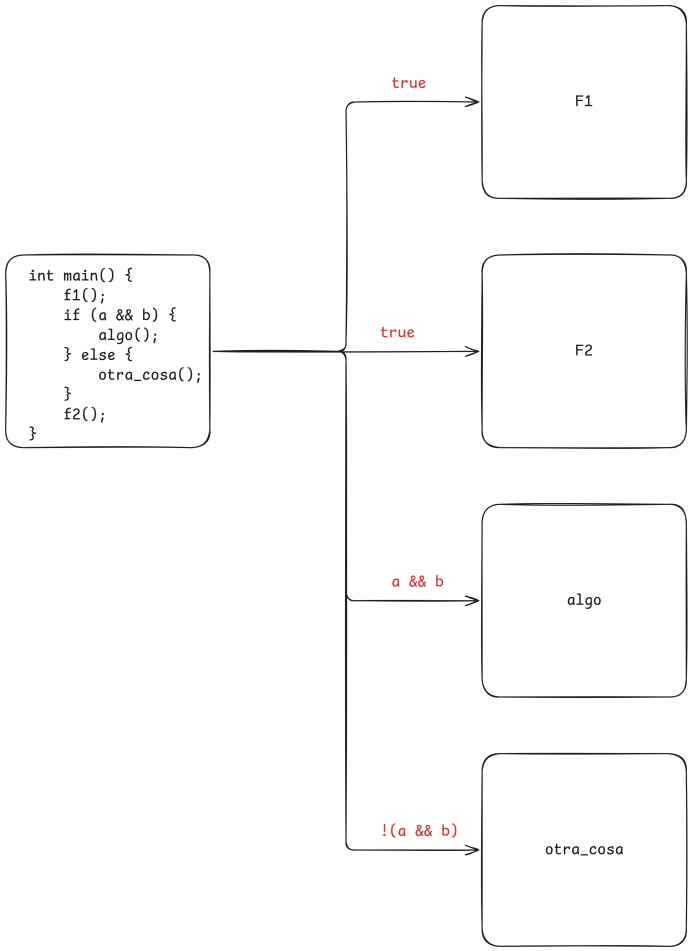


Figura 8: Árbol de ejecución de un programa simple. Nótese como cada función genera su propio bloque y algunas ramas tienen condiciones que deben cumplirse para ser ejecutadas.

Éste árbol es justamente lo que programas como AFL++ exploran y es la segunda estrategia que usaron en SQLite para aliviar los bugs [29]. Su objetivo es variar de forma automática varios inputs del programa, lo que a su vez varía las condicinoes de ejecución de ciertas ramas, reportando solamente las ramas que hacen que el programa crashee, lo que a su vez ayuda a descubrir caminos que rara vez son ejecutados normalmente [31]. Hay dos tipos principales de *fuzzing*:

- Mutacional: La campaña de fuzzing inicia con varias «semillas». Luego estas semillas son modificadas para generar nuevas entradas al programa.
- Generacional: Las entradas son generadas desde 0, si se especifica un formato entonces se generan entradas que cumplan ese formato. Solamente las entradas que más profundo lleguen a explorar el sistema son evolucionadas a la siguiente ronda [31].

4.6.1.2 LLMs

Richar Hipp menciona que recientemente, a pesar de todos los pasos que ha tomado para asegurarse que el código de SQLite es lo más libre de bugs posible, los LLMs han encontrado formas exóticas de ejecución de código que revelan vulnerabilidades en su código [29].

Por lo tanto los LLMs deben ser considerados, cuanto mínimo, como un linter más que pueden ayudar a revisar el código y proponer mejoras. Curiosamente, este uso no se alinea más a las propiedades innatas de ellos, siendo estocásticos y alucinando de vez en cuando.

V

Metodología

5.1 Ambiente de Desarrollo

Para el desarrollo de las bases del *software* de vuelo de la computadora a bordo (OBC por sus siglas en inglés) secundaria del Quetzal-2 se utilizó el sistema operativo libre Linux. Además se necesita:

1. Un editor de código.
2. El manejador de paquetes Nix (versión 2.34.8) con Nix Flakes habilitados.
3. El STM32CubeIDE (versión 2.2.0), para generar la configuración del hardware.
4. OpenMV IDE, para interactuar con la cámara de OpenMV de *Payload* MILO.

El archivo `flake.nix` del repositorio (se encuentra en la Sección 9.1) lista todas las dependencias que el proyecto necesita a detalle, sin embargo, una lista con las principales es la siguiente:

- gcc v15.2.0
- make v4.4.1
- bear v4.0.3

Los pasos para compilar el proyecto desde 0 se pueden encontrar en detalle en el README del repositorio (Sección 9.1).

5.2 Configuración del Hardware

La OBC secundaria se encuentra compuesta por 2 *PortentasH7 Lite*. Ambas se configuraron utilizando el STM32H7CubeIDE para autogenerar el código de configuración. Específicamente, la OBC secundaria utiliza 2 tipos de comunicación serial: I2C y UART. Se configuraron de la siguiente manera:

5.2.1 I2C

La OBC utiliza la comunicación I2C únicamente entre la computadora primaria y la secundaria diseñada localmente:

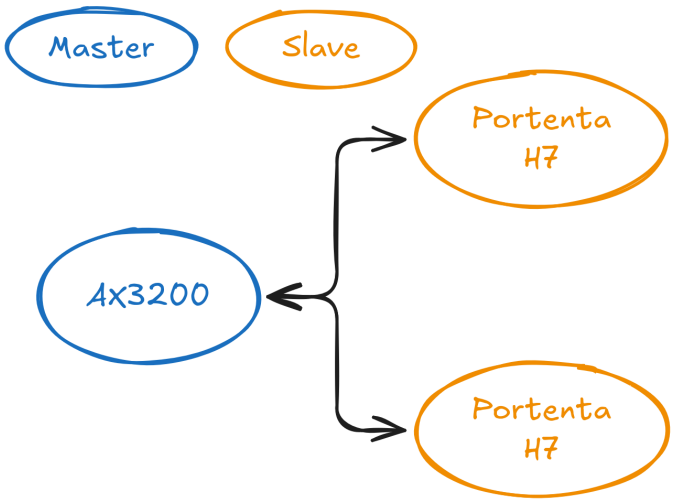


Figura 9: Comunicación I2C entre computadora primaria y secundaria.

Para esta comunicación se utiliza el puerto I2C1 con la siguiente configuración:

```
I2C_HandleTypeDef hi2c1;

void MX_I2C1_Init(void) {

  hi2c1.Instance = I2C1;
  hi2c1.Init.Timing = 0x10C0ECFF;
  hi2c1.Init.OwnAddress1 = (0x09 << 1);
  hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2 = 0;
  hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c1) != HAL_OK) {
    Error_Handler();
  }
  /** Configure Analogue filter
   */
  if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK) {
    Error_Handler();
  }
  /** Configure Digital filter
   */
  if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK) {
    Error_Handler();
  }
}
```

5.2.2 UART

La comunicación UART se utiliza para la comunicación entre la OBC y los demás módulos del satélite, por ejemplo con ADCS, MILO, etc.

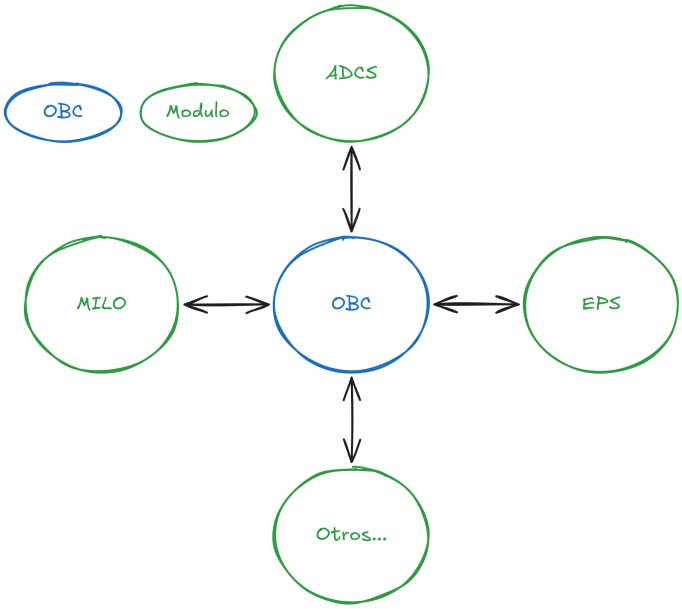


Figura 10: Comunicación UART entre OBC y módulos del satélite.

Para la comunicación utilizando RS485, se utiliza un puerto en específico de UART: El UART4. Se inicializa de la siguiente manera:

```
void MX_UART4_Init(void) {
  huart4.Instance = UART4;
  huart4.Init.BaudRate = 115200;
  huart4.Init.WordLength = UART_WORDLENGTH_8B;
  huart4.Init.StopBits = UART_STOPBITS_1;
  huart4.Init.Parity = UART_PARITY_NONE;
  huart4.Init.Mode = UART_MODE_TX_RX;
  huart4.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart4.Init.OverSampling = UART_OVERSAMPLING_16;
  huart4.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
  huart4.Init.ClockPrescaler = UART_PRESCALER_DIV1;
  huart4.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
  if (HAL_UART_Init(&huart4) != HAL_OK) {
    Error_Handler();
  }
  if (HAL_UARTEx_SetTxFifoThreshold(&huart4, UART_TXFIFO_THRESHOLD_1_8) !=
      HAL_OK) {
    Error_Handler();
  }
  if (HAL_UARTEx_SetRxFifoThreshold(&huart4, UART_RXFIFO_THRESHOLD_1_8) !=
      HAL_OK) {
    Error_Handler();
  }
  if (HAL_UARTEx_DisableFifoMode(&huart4) != HAL_OK) {
    Error_Handler();
```

```
  }
}
```

La OBC secundaria también utiliza un puerto UART para enviar *logs* con información adicional que aumentan la observabilidad detrás de qué está pasando internamente dentro del sistema, los cuales se pueden leer leyendo el puerto UART6, configurado de la siguiente manera:

```
void MX_USART6_UART_Init(void) {
  huart6.Instance = USART6;
  huart6.Init.BaudRate = 115200;
  huart6.Init.WordLength = UART_WORDLENGTH_8B;
  huart6.Init.StopBits = UART_STOPBITS_1;
  huart6.Init.Parity = UART_PARITY_NONE;
  huart6.Init.Mode = UART_MODE_TX_RX;
  huart6.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart6.Init.OverSampling = UART_OVERSAMPLING_16;
  huart6.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
  huart6.Init.ClockPrescaler = UART_PRESCALER_DIV1;
  huart6.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
  if (HAL_UART_Init(&huart6) != HAL_OK) {
    Error_Handler();
  }
  if (HAL_UARTEx_SetTxFifoThreshold(&huart6, UART_TXFIFO_THRESHOLD_1_8) !=
      HAL_OK) {
    Error_Handler();
  }
  if (HAL_UARTEx_SetRxFifoThreshold(&huart6, UART_RXFIFO_THRESHOLD_1_8) !=
      HAL_OK) {
    Error_Handler();
  }
  if (HAL_UARTEx_DisableFifoMode(&huart6) != HAL_OK) {
    Error_Handler();
  }
}
```

5.3 OBC Secundaria Quetzal-2

La OBC secundaria se diseñó a nivel de *software* para cumplir los siguientes objetivos, en orden de prioridad:

1. Fiabilidad
2. Seguridad
3. Rendimiento

La OBC secundaria se implementó como una máquina de estados finitos determinista, de esta forma podemos predecir no solo el rendimiento de memoria de la computadora secundaria, sino también su comportamiento. Algunos de los estados de la máquina finita se pueden apreciar en la Figura 11.

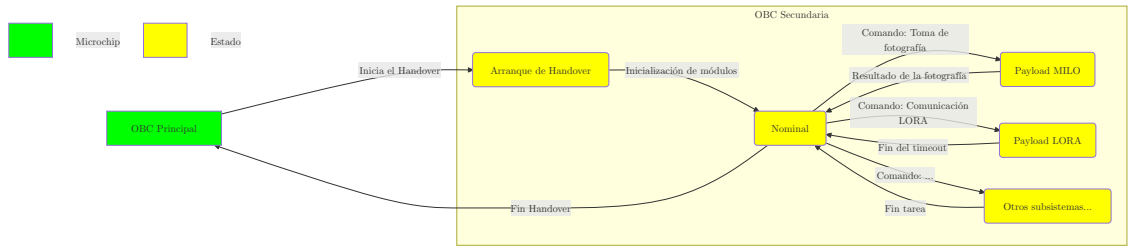


Figura 11: Máquina de Estados Finitos (incompleta) de la OBC secundaria del Quetzal-2. Elaboración propia.

Para mejorar el determinismo de esta máquina de estados, todas las tareas que necesitan comunicación con un sistema externo al *core* de la OBC secundaria, por ejemplo interactuar con subsistemas, se realizaron por medio de interfaces. De esta forma se tiene una arquitectura como la descrita en la Figura 12.

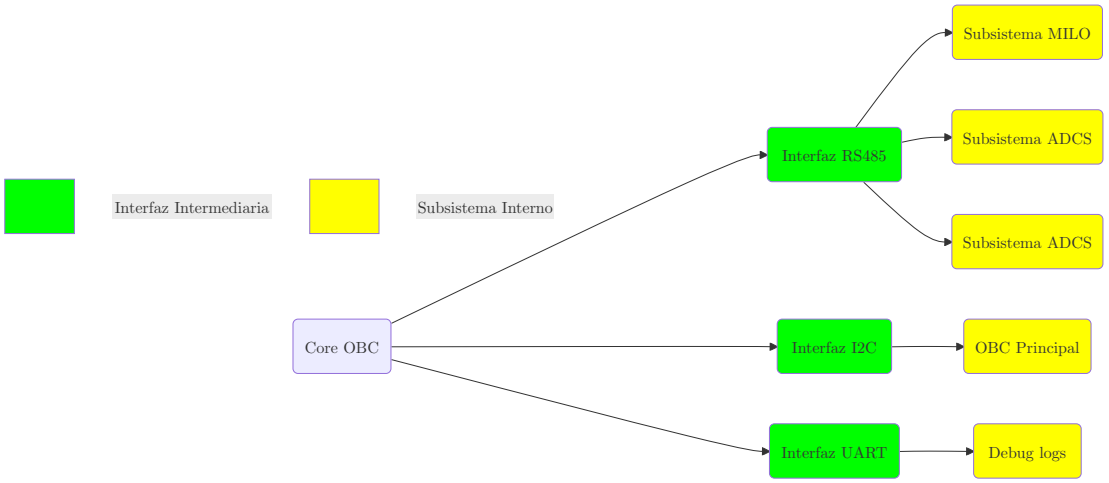


Figura 12: Arquitectura de la OBC secundaria. Elaboración propia.

Esta arquitectura de la Figura 12 nos permitió varias bondades:

- Nos permitió intercambiar todos los subsistemas (en amarillo) por implementaciones que funcionen o fallen de formas predecibles.
- Se logró simular entradas y salidas de los distintos subsistemas a voluntad sin necesidad de conectar físicamente todo el sistema o que siquiera se encuentre desarrollado.
- Se automatizaron las pruebas de comportamiento de la OBC ante una gran variedad de entradas de los distintos submódulos.
- Automatización de generación de casos de prueba con *fuzzing* en AFL++.

VI

Resultados

```
================================================
Testing: obc_test.c
================================================
+ gcc ../../deps/unity/unity.o obc_test.c -lm -DQTZ_DEBUG -g -o test.bin
+ ./test.bin
../obc.c:296  [LOG] [OBC-MR]: State: [IDLE                  ][MILO_UNSTARTED          ], received command: [H][0][P][PING              ][0][0]
../obc.c:81   [LOG] [OBC-MR]: State: [IDLE                  ][MILO_UNSTARTED          ], sending  command: [H][0][G][PING_ACK          ][0][0]
obc_test.c:391:test_QTZ_BasicPing:PASS
../obc.c:296  [LOG] [OBC-MR]: State: [IDLE                  ][MILO_UNSTARTED          ], received command: [H][0][P][PING              ][0][0]
../obc.c:81   [LOG] [OBC-MR]: State: [IDLE                  ][MILO_UNSTARTED          ], sending  command: [H][0][G][PING_ACK          ][0][0]
../obc.c:296  [LOG] [OBC-MR]: State: [IDLE                  ][MILO_UNSTARTED          ], received command: [H][0][P][BEGIN_HANDOVER    ][0][0]
../obc.c:100  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], sending  command: [H][0][G][HANDOVER_BEGIN_ACK][0][0]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], received command: [H][0][P][HEARTBEAT         ][0][0]
../obc.c:117  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], sending  command: [H][0][G][HEARTBEAT_ACK     ][0][0]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], received command: [H][0][P][HEARTBEAT         ][0][0]
../obc.c:117  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], sending  command: [H][0][G][HEARTBEAT_ACK     ][0][0]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], received command: [H][0][P][STATUS            ][0][0]
../obc.c:128  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], sending  command: [H][0][G][STATUS_ACK        ][2][0]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_UNSTARTED          ], received command: [H][0][P][START_TASK        ][0][0]
../obc.c:157  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_BEGIN              ], sending  command: [H][0][G][START_TASK_ACK    ][0][0]
../obc.c:167  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_BEGIN              ], sending  command: [S][0][M][PING              ][0][0]
../obc.c:285  [WAR] [OBC-MR]: No packet received from gomspace! Checking other submodules...
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_BEGIN              ], received command: [S][0][P][PING_ACK          ][0][0]
../obc.c:226  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_TAKE_PICTURE       ], sending  command: [S][0][M][TAKE_PICTURE      ][0][0]
../obc.c:285  [WAR] [OBC-MR]: No packet received from gomspace! Checking other submodules...
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_TAKE_PICTURE       ], received command: [S][0][P][TAKE_PICTURE_ACK  ][0][0]
../obc.c:246  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_GET_DATA           ], sending  command: [S][0][M][PICTURE_CLASI     ][0][0]
../obc.c:285  [WAR] [OBC-MR]: No packet received from gomspace! Checking other submodules...
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_GET_DATA           ], received command: [S][0][P][PICTURE_CLASI_ACK ][3][0]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_END                ], received command: [H][0][P][STATUS            ][0][0]
../obc.c:128  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_END                ], sending  command: [H][0][G][STATUS_ACK        ][2][4]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_END                ], received command: [H][0][P][GET_IMAGE_CLASI   ][0][0]
../obc.c:178  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_END                ], sending  command: [H][0][G][GET_IMAGE_CLASI_ACK][3][0]
../obc.c:296  [LOG] [OBC-MR]: State: [HANDOVER_IDLE         ][MILO_END                ], received command: [H][0][P][ABORT_HANDOVER    ][0][0]
../obc.c:198  [LOG] [OBC-MR]: State: [IDLE                  ][MILO_END                ], sending  command: [H][0][G][ABORT_HANDOVER_ACK][0][0]
obc_test.c:392:test_QTZ_HandoverInit:PASS
```

Figura 13: Prueba del protocolo de handover corriendo localmente dentro de una misma computadora.



Figura 14: Prueba del protocolo de handover corriendo con hardware real entre la computadora primaria y secundaria. *Logs* desde la perspectiva de la OBC secundaria.



Figura 15: Prueba del protocolo de handover corriendo con hardware real entre la computadora primaria y secundaria. Realizado durante la visita de David Cuartielles, cofundador de Arduino y Andrés Sabas, fundador de Electronic Cats.



Figura 16: Fotografía tomada con la *Payload MILO* durante la prueba de handover.

VII

Discusión de Resultados

Como se puede apreciar en la Figura 13, se realiza el camino sin errores de una rutina de handover completa, desde que la OBC primaria solicita el traslado de poder, pasando por el inicio de la tarea de MILO en donde se toma la fotografía y finalizando con el comando de abortar de la OBC primaria.

Toda esa rutina se ejecutó dentro de un ambiente simulado en la computadora, y cada una de las respuestas y acciones que decidía tomar la OBC secundaria fue analizada por la prueba, solamente retornando éxito si todos los comandos recibidos y enviados seguían la especificación del protocolo interno. Es decir, la prueba revisó que luego de recibir el comando `BEGIN_HANDOVER` desde la computadora principal, la OBC secundaria respondiera con `BEGIN_HANDOVER_ACK`.

Nótese el cambio de estados dentro de la OBC secundaria que se puede apreciar en los *logs*, iniciando en `IDLE` para transicionar a `HANDOVER_IDLE` y finalmente a `IDLE` de nuevo. Estos son los estados principales de la OBC secundaria, mientras que el estado interno de la tarea es `MILO_UNSTARTED`. La combinación de estos dos estados (`IDLE`, `MILO_UNSTARTED`) representan el modo de operación de arranque dentro del Producto Mínimo Viable (MVP por sus siglas en inglés) de la OBC secundaria. Para luego transicionar al estado nominal (`HANDOVER_IDLE`, `MILO_UNSTARTED`), para finalmente llegar al estado de toma de fotografía (`HANDOVER_IDLE`, `MILO_BEGIN`). La Figura 17 muestra en forma de diagrama estas transiciones:

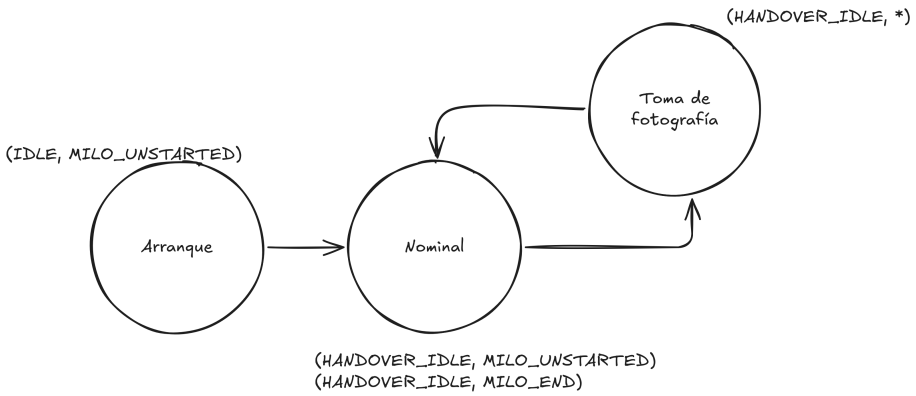


Figura 17: Estados de la OBC secundaria de estados finitos.

Esta es la prueba principal que se implementó, pero gracias a la arquitectura hexagonal se tiene la posibilidad de implementar muchas más que prueben casos más extraños, o simplemente probar casos de error que permitan asegurar que la OBC fallará de una manera predecible cuando se cumplan ciertas condiciones.

El que la prueba mostrada en la Figura 13 tenga un resultado satisfactorio no significa que la OBC secundaria funcione, puesto que es un ambiente simulado dentro de la misma computadora. Por esto se realizaron pruebas con implementaciones reales de las interfaces, las cuales se pueden ver en la Figura 14 y la Figura 15. En esta prueba se intentó solamente la comunicación entre los dos módulos de la OBC, primaria y secundaria. Como se puede ver en los *logs*, la comunicación fue satisfactoria y se logró llegar al último comando de aborto del protocolo de handover.

Finalmente, en la Figura 16 se puede apreciar la fotografía de una prueba final con todos los subsistemas conectados utilizando las implementaciones reales de las interfaces de comunicación. La imagen se encuentra borrosa debido a la configuración de la cámara, ya que se tenía en baja calidad para limitar la cantidad de espacio que cada fotografía consumía al ser almacenada y transmitida dentro del sistema. Para ese punto de las pruebas lo importante no era la calidad de las fotografías sino que el sistema pudiera tomarla de forma autónoma.

VIII

Conclusiones

1. Se integró el módulo de cámara el sistema de *software* de la computadora a bordo (OBC por sus siglas en inglés) secundaria, tomando una fotografía que se puede apreciar en la Figura 16.
2. Se desarrolló un sistema de gestión de modos de operación que permite: Arranque, Nominal preliminar y Toma de fotografía (Figura 13).
3. Se desarrolló un producto mínimo viable del sistema de *handover* entre las dos OBCs del satélite (Figura 15).

Bibliografía

[1] U. del Valle de Guatemala, «Satélite CubeSat Quetzal-2». [En línea]. Disponible en: https://www.uvg.edu.gt/quetzal-2/

[2] UVG, «CubeSat». [En línea]. Disponible en: https://www.uvg.edu.gt/cubesat/

[3] A. Johnstone, *CubeSat Design Specification Rev. 14.1 The CubeSat Program, Cal Poly SLO CubeSat Design Specification Cal Poly -San Luis Obispo, CA Document Classification X Public Domain*. 2022. [En línea]. Disponible en: https://static1.squarespace.com/static/5418c831e4b0fa4ecac1bacd/t/62193b7fc9e72e0053f00910/1645820809779/CDS+REV14_1+2022-02-09.pdf

[4] T. Lwabanji, R. Wilkinson, y E. Biermann Bellville, *Development of an OnBoard Computer (OBC) for a CubeSat*. 2013. [En línea]. Disponible en: https://etd.cput.ac.za/bitstream/20.500.11838/1172/1/Lumbwe_T_Final2013.pdf

[5] S. A. Akhtar, *Design and Development of Obc of a Pico Spacecraft*. LAP Lambert Academic Publishing, 2012.

[6] S. R. Hirshorn, *NASA Systems Engineering Handbook*. 2007. [En línea]. Disponible en: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf?emrc=4096b9

[7] B. Rajulu, S. Dasiga, y N. R. Iyer, «Open source RTOS implementation for on-board computer (OBC) in STUDSAT-2», *IEEE*, mar. 2014, doi: https://doi.org/10.1109/aero.2014.6836377.

[8] Arduino, *Arduino® Portenta H7 Collective Datasheet*. 2026. [En línea]. Disponible en: https://docs.arduino.cc/resources/datasheets/ABX00042-ABX00045-ABX00046--datasheet.pdf

[9] FreeRTOS, «FreeRTOS - Market Leading RTOS (Real Time Operating System) for Embedded Systems with Internet of Things Extensions». [En línea]. Disponible en: https://www.freertos.org/

[10] D. Gildeh, *Final Report: Design and development of OBC for Pico-Sat PALMSAT*. 2003. [En línea]. Disponible en: https://davidgildeh.com/sites/davidgildeh.com/files/projects/Palmsat_Report.pdf

[11] A. Aguilar-Nadalini *et al.*, «Design and On-Orbit Performance of the Electrical Power System for the Quetzal-1 CubeSat», *JoSS*, vol. 12, n.º 02, p. 1201, 2023, [En línea]. Disponible en: https://jossonline.com/storage/2023/05/Final-Aguilar-Nadalini-Design-and-On-Orbit-Performance-of-the-Electrical-Power-System-for-the-Quetzal-1-CubeSat.pdf

[12] Gomspace, *NanoMind A3200 - GomSpace*. 2026. [En línea]. Disponible en: https://gomspace.com/product/nanomind-a3200/

[13] L. Stras *et al.*, «The design and operation of the Canadian advanced nanospace eXperiment (CanX-1)», en *Proc. AMSAT-NA 21st Space Symposium, Toronto, Canada*, 2003, pp. 150-160.

[14] N. D. Hidayat, *Development of Flight Software for The Nanosatellite Onboard Computer Based on FM430 and Real-Time Operating System*. 2010.

[15] P. D. Hung, V. Van Chin, N. T. Chinh, y T. D. Tung, «A Flexible Platform for Industrial Applications Based on RS485 Networks.», *J. Commun.*, vol. 15, n.º 3, pp. 245-255, 2020.

[16] J. Sastry, A. Suresh, y S. Bhanu, «Building heterogeneous distributed embedded systems through rs485 communication protocol», *ARPN Journal of engineering and applied sciences*, vol. 10, n.º 16, pp. 6793-6803, 2015.

[17] E. J. Carletti, «Comunicación-bus i2c», *Robots Argentina*, 2007.

[18] E. Peña y M. G. Legaspi, «Uart: A hardware communication protocol understanding universal asynchronous receiver/transmitter», *Visit Analog*, vol. 54, n.º 4, pp. 1-5, 2020.

[19] J. Sonnenberg, «Serial Communications RS232, RS485, RS422», 2018.

[20] R. Schwarz y S. Theil, «A Fault-Tolerant On-Board Computing and Data Handling Architecture Incorporating a Concept for Failure Detection, Isolation, and Recovery for the SHEFEX III Navigation System», en *SpaceOps 2014 Conference*, 2014, p. 1874.

[21] J.-F. Soucaille, «I. High Dependability Data Handling Systems for In Orbit Operations», en *2023 European Data Handling & Data Processing Conference (EDHPC)*, 2023, pp. 1-4.

[22] D. M. Marcos y A. Valverde Carretero, «DHS architecture for HERA Deep Space Mission». [En línea]. Disponible en: http://dx.doi.org/10.23919/EDHPC59100.2023.10396073

[23] STMicroelectronics, *STM32H7 - Arm Cortex-M7 and Cortex-M4 MCUs (480 MHz) - STMicroelectronics*. 2026. [En línea]. Disponible en: https://www.st.com/en/microcontrollers-microprocessors/stm32h7-series.html

[24] Abraham Silberschatz; Greg Gagne; Peter B. Galvin, *Operating System Concepts*. Wiley Global Education, 2018.

[25] O. Schuele, M. Rafat, y V. Kossykh, «The software environment of RODOS», pp. p. 1109-1118, 1996.

[26] C. B. Pareja Valerio y L. J. Burgos Robles, «La arquitectura de software basada en microservicios: Una revisión sistemática de la literatura», 2019.

[27] G. Holzmann, «The Power of 10: Rules for Developing Safety-Critical Code», en *Software Technologies*, 2006. [En línea]. Disponible en: https://web.eecs.umich.edu/~imarkov/10rules.pdf

[28] J. D. Greef, «A Trillion Transactions». [En línea]. Disponible en: https://tigerbeetle.com/blog/2026-03-19-a-trillion-transactions/

[29] S. S. Work, «Reliability Lessons From SQLite - Richard Hipp | SSW 2026». [En línea]. Disponible en: https://www.youtube.com/watch?v=V_qzqY1bb7I

[30] A. Elmasry y J. Katajainen, «Branchless search programs», en *International Symposium on Experimental Algorithms*, 2013, pp. 127-138.

[31] M. Böhme, C. Cadar, y A. Roychoudhury, «Fuzzing: Challenges and reflections», *IEEE Software*, vol. 38, n.º 3, pp. 79-86, 2020.

IX

Anexos

9.1 Repositorio de Código

El repositorio es público y se encuentra hosteado en github, da click aquí.

Glosario

Latex
Es un lenguaje de marcado adecuado especialmente para la creación de documentos científicos

Fórmula
Una expresión matemática